Object Oriented Programming I

September 18, 2024

Content

Time Class

Class Scope and Accessing Class Members

Access Functions and Utility Functions

Constructors with Default Arguments

Destructors

Time Class Definition

```
1  /* ****************************************
2   * Filename     :   Time.h
3   * Author       :   Burak Kaleci
4   * Date         :   10.10.2018
5   * Description  :   An example program
6   * ****************************************
7
8  // prevent multiple inclusions of header
9  #ifndef TIME_H
10 #define TIME_H
11
12 // Time class definition
13 class Time
14 {
15 public:
16
17     /// Constructor
18     Time();
19     /// Set hour, minute and second
20     void setTime(int,int,int);
21     /// Print time in universal-time format
22     void printUniversal(void);
23     /// Print time in standard-time format
24     void printStandard(void);
25
26 private:
27
28     int hour; // 0 - 23
29     int minute; // 0 - 59
30     int second; // 0 - 59
31 };
32
33 #endif
```

- The class definition contains prototypes (lines 18-24) for member functions Time, setTime, printUniversal and printStandard.
- It includes private integer members hour, minute and second (lines 28-30).
- Class Time's private data members can be accessed only by its four public member functions.

Preprocessor Wrapper

- The class definition is enclosed in the following preprocessor wrapper (lines 9, 10 and 33):

```
#ifndef TIME_H
#define TIME_H
...
#endif
```

- When we build larger programs,other definitions and declarations will also be placed in headers.
- The preceding preprocessor wrapper prevents the code between #ifndef (which means "if not defined") and #endif from being included if the name TIME_H has been defined.
- If the header has not been included previously in a file,the name TIME_H is defined by the #define directive and the header statements are included.
- If the header has been included previously, TIME_H is defined already and the header is not included again.
- Attempts to include a header multiple times (inadvertently) typically occur in large programs with many headers that may themselves include other headers.

Time Class Member Functions

```cpp
/* ******************************************************
 * Filename     :   Time.cpp
 * Author       :   Burak Kaleci
 * Date         :   10.10.2018
 * Description  :   Time Class Implementation
 * ******************************************************/

#include <iostream>     // for cout
#include <iomanip>      // for setfill and setw
#include <stdexcept>    // for invalid_argument exception class

#include "Time.h"   // include definition of class Time

using namespace std;

/// Time constructor initializes each data member to zero.
Time::Time()
{
    hour = minute = second = 0;
}

/// set new Time value using universal time
void Time::setTime(int h ,int m ,int s)
{
    // validate hour, minute and second
    if ((h>=0 && h<24) && (m>=0 && m<60) && (s>= 0 && s< 60)){
        hour =h;
        minute =m;
        second =s;
    }
    else
        throw invalid_argument("hour,minute and/or second was out of range");
}
```

```cpp
/// print Time in universal-time format (HH:MM:SS)
void Time::printUniversal(void)
{
    cout << setfill('0') << setw( 2 ) << hour << ":"
        << setw( 2 )<< minute << ":" << setw( 2 ) << second;
}

// print Time in standard-time format (HH:MM:SS AM or PM)
void Time::printStandard(void)
{
    if (hour == 0 || hour == 12 )
        cout << 12;
    else
        cout << hour % 12;

    cout << ":" << setfill('0') << setw( 2 )<< minute << ":" << setw( 2 )
        << second;

    if (hour < 12)
        cout << " AM";
    else
        cout << " PM";
}
```

Time Class Member Functions

- The Time constructor (lines 18-21) initializes the data members to 0.
- Invalid values cannot be stored in the data members of a Time object, because the constructor is called when the Time object is created,and all subsequent attempts by a client to modify the data members are examined diligently by function setTime.
- With the exception of **static const int** data members, a class's data members cannot be initialized where they're declared in the class body.
- In other words, attempting to initialize a non-static data member of a class explicitly in the class definition (Interface or Header code) is a syntax error.
- It's strongly recommended that these data members be initialized by the class's constructor. Data members can also be assigned values by Time's set functions.

setTime Member Functions

- Function setTime (lines 24-34) is a public function that declares three int parameters and uses them to set the time.
- Line 27 tests each argument to determine whether the value is in range,and,if so, lines 28-30 assign the values to the hour, minute and second data members.
- The hour value must be greater than or equal to 0 and less than 24,because universal-time format represents hours as integers from 0 to 23.
- Similarly, both minute and second must be greater than or equal to 0 and less than 60.

Exception Handling

- An **exception** indicates a problem that occurs while a program executes.
- The name "exception" suggests that the problem occurs infrequently which means that if the "rule" is that a statement normally executes correctly, then the problem represents the "exception to the rule."
- Exception handling enables you to create fault-tolerant programs that can resolve (or handle) exceptions.
- In many cases,this allows a program to continue executing as if no problems were encountered.
- More severe problems might prevent a program from continuing normal execution,instead requiring the program to notify the user of the problem, then terminate.
- When a function detects a problem, such as an invalid data members, it throws an exception that is, an exception occurs.

Exception Handling

- For values outside the valid ranges setTime throws an exception of type invalid_argument (line 33),which notifies the client code that an invalid argument was received.
- The throw statement (line 33) creates a new object of type invalid_argument.
- The parentheses following the class name indicate a call to the invalid_argument constructor that allows us to specify a custom error message string.
- After the exception object is created, the throw statement immediately terminates function setTime and the exception is returned to the code that attempted to set the time.
- To avoid compilation errors, you may need to include header <stdexcept> to use class invalid_argument.

Exception Handling

```
28      // attempt to set the time with invalid values
29      try
30      {
31          t.setTime( 99, 99, 99 ); // all values out of range
32      }
33      catch (invalid_argument &e)
34      {
35          cout << "\nException: " << e.what() << endl << endl;
36      }
```

- To illustrate that method setTime validates its arguments, line 31 calls setTime with invalid arguments of 99 for the hour, minute and second.
- To handle an exception, place any code that might throw an exception in a try statement (lines 29-36).
- The try block (lines 29-32)contains the code that might throw an exception, and the catch block (lines 33-36) contains the code that handles the exception if one occurs.
- You can have many catch blocks to handle different types of exceptions that might be thrown in the corresponding try blocks.
- The braces that delimit try and catch blocks' bodies are required.

Exception Handling

```
28      // attempt to set the time with invalid values
29      try
30      {
31          t.setTime( 99, 99, 99 ); // all values out of range
32      }
33      catch (invalid_argument &e)
34      {
35          cout << "\nException: " << e.what() << endl << endl;
36      }
```

- Line 31 is placed in a try block (lines 29-32) in case setTime throws an invalid_argument exception.
- When setTime throws an invalid_argument exception, the try block terminates immediately and the catch block begins executing.
- If you declared any variables in the try block, they are now out of scope and are not accessible in the catch block.
- Line 35 displays the exception's error message by calling its what member function.
- what member function is used to get the error message that is stored in the exception object (in this example e) and display it.
- Once the message is displayed, the exception is considered handled and the program continues with the next statement after the catch block's closing brace.
- The setTime does not change data members when invalid arguments are supplied.
- If the code in the try block executes successfully, lines 33-36 are ignored.

printUniversal Member Functions

- Function printUniversal takes no arguments and outputs the time in universal-time format,consisting of three colon-separated pairs of digits for the hour, minute and second.
- For example, if the time were 1:30:07 PM, function printUniversal would return 13:30:07.
- Line 39 uses parameterized stream manipulator setfill to specify the fill character that's displayed when an integer is output in a field wider than the number of digits in the value.
- By default,the fill characters appear to the left of the digits in the number. In this example, if the minute value is 2,it will be displayed as 02, because the fill character is set to zero('0').
- If the number being output fills the specified field,the fill character will not be displayed.
- Once the fill character is specified with setfill, it applies for all subsequent values that are displayed in fields wider than the value being displayed.
- This is in contrast to setw, which applies only to the next value displayed.

printStandard Member Functions

- Function printStandard takes no arguments and outputs the time in standard-time format, consisting of the hour, minute and second values separated by colons and followed by an AM or PM indicator.
- Like function printUniversal,function printStandard uses setfill('0') to format the minute and second as two digit values with leading zeros if necessary.
- Lines 46-49 determine the value of hour to be displayed. If the hour is 0 or 12, it appears as 12; otherwise ,the hour appears as a value from 1 to 11.
- Lines 54-57 determine whether AM or PM will be displayed.

Member Functions vs. Global Functions

- The printUniversal and printStandard member functions take no arguments and return nothing, because these member functions implicitly know that they're to print the data members of the particular Time object for which they're invoked.
- This can make member function calls more brief than conventional function calls in procedural programming.
- Using an object-oriented programming approach often simplifies function calls by reducing the number of parameters.
- This benefit of object-oriented programming derives from the fact that encapsulating data members and member functions within an object gives the member functions the right to access the data members.

Using Class Time

```cpp
/* ********************************************************
 * Filename     :   main.cpp
 * Author       :   Burak Kaleci
 * Date         :   10.10.2018
 * Description  :   Driver program for class Time
 * ********************************************************/

#include <iostream>
#include <stdexcept>
#include "Time.h"   // include definition of class Time

using namespace std;

int main() {

    Time t;     // instantiate object t of class Time

    cout << "The initial universal time is ";
    t.printUniversal(); // 00:00:00

    cout << "\nThe initial standard time is ";
    t.printStandard(); // 12:00:00 AM

    t.setTime(13, 27, 6); // change time

    // output Time object t's new values
    cout << "\n\nUniversal time after setTime is ";
    t.printUniversal(); // 13:27:06

    cout << "\nStandard time after setTime is ";
    t.printStandard(); // 1:27:06 PM

    // attempt to set the time with invalid values
    try
    {
        t.setTime( 99, 99, 99 ); // all values out of range
    }
    catch (invalid_argument &e)
    {
        cout << "\nException: " << e.what() << endl << endl;
    }

    // output t's values after specifying invalid values
    cout << "\n\nAfter attempting invalidsettings:"
        << "\nUniversal time:";
        t.printUniversal(); // 00:00:00

    cout << "\nStandard time: ";
    t.printStandard(); // 12:00:00 AM
    cout << endl;

    return 0;
}
```

Class Scope

- With in a class's scope, class members are immediately accessible by all of that class's member functions and can be referenced by name.
- Outside a class's scope, public class members are referenced through one of the handles on an object:
  - An object name (Time t)
  - A reference to an object (Time &tRef=t)
  - A pointer to an object (Time *timePtr= &tRef)

  The type of the object,reference or pointer specifies the interface(i.e.,the member functions) accessible to the client.

Accessing Class Members

```
1  /* ********************************************************
2   * Filename     :   accessOperator.cpp
3   * Author       :   Burak Kaleci
4   * Date         :   27.09.2018
5   * Description  :   An example program to introduce access operators
6   * ********************************************************/
7
8   // includes input/output stream header
9  // To output data to the screen
10 #include <iostream>
11 using namespace std;
12
13 class Count
14 {
15 public:
16
17     // sets thevalue of privatedatamember x
18     void setX( int value )
19     {
20         x= value;
21     }
22
23     // prints the valueofprivate data member x
24     void print()
25     {
26         cout << x<<endl;
27     }
28
29 private:
30     int x;
31 };
```

The following code uses a simple class called Count (lines 13-31)with private data member x of type int (line 30), public member function setX (lines 18-21) and public member function print (lines 24-27) to illustrate accessing class members with the member-selection operators.

Accessing Class Members

```
33  main()
34  {
35      Count counter; // create counter object
36      Count *counterPtr =&counter; // create pointer to counter
37      Count &counterRef =counter; // create reference to counter
38
39      cout << "Set x to 1 and print using object's name: ";
40      counter.setX( 1 ); // set data member x to 1
41      counter.print(); // call member function print
42
43      cout << "Set x to 2 and print using a reference to an object: ";
44      counterRef.setX( 2 ); // set data member x to 2
45      counterRef.print(); // call member function print
46
47      cout << "Set x to 3 and print using a pointer to an object: ";
48      counterPtr->setX( 3 ); // set data member x to 3
49      counterPtr->print(); // call member function print
50  }
51
```

- The dot member selection operator (.) is preceded by **an object's name** or **with a reference to an object** to access the object's members.
- The arrow member selection operator ($->$) is preceded by **a pointer to an object** to access the object's members.
- Lines 35-37 create three variables related to type Count-counter (a Count object), counterPtr (a pointer to a Count object) and counterRef (a reference to a Count object).
- Variable counterRef refers to counter,and variable counterPtr points to counter.

Accessing Class Members

```
33  main()
34  {
35      Count counter; // create counter object
36      Count *counterPtr =&counter; // create pointer to counter
37      Count &counterRef =counter; // create reference to counter
38
39      cout << "Set x to 1 and print using object's name: ";
40      counter.setX( 1 ); // set data member x to 1
41      counter.print(); // call member function print
42
43      cout << "Set x to 2 and print using a reference to an object: ";
44      counterRef.setX( 2 ); // set data member x to 2
45      counterRef.print(); // call member function print
46
47      cout << "Set x to 3 and print using a pointer to an object: ";
48      counterPtr->setX( 3 ); // set data member x to 3
49      counterPtr->print(); // call member function print
50  }
51
```

- In lines 40-41 and 44-45,note that the program invokes member functions setX and print by using the dot (.)member selection operator preceded by either the name of the object (counter) or a reference to the object(counterRef).
- Lines 48-49 demonstrate that the program can invoke member functions setX and print by using a pointer (countPtr) and the arrow ($->$) member-selection operator.

Access Functions

- Access functions can read or display data. Another common use for access functions is to test the truth or falsity of conditions such functions are often called predicate functions.
- An example of a predicate function would be an isEmpty function for any container class(a class capable of holding many objects,like a vector).
- A program might test isEmpty before attempting to read another item from the container object.
- An isFull predicate function might test a container-class object to determine whether it has no additional room.
- Useful predicate functions for our Time class might be isAM and isPM.

Access Functions

- A utility function (also called a helper function) is not part of a class's public interface; rather, **it's a private member function** that supports the operation of the class's other member functions.
- Utility functions are not intended to be used by clients of a class.

SalesPerson Class Definition

```
1  /* ************************************
2   * Filename     :   SalesPerson.h
3   * Author       :   Burak Kaleci
4   * Date         :   10.10.2018
5   * Description  :   An example program
6   * ************************************
7
8  // prevent multiple inclusions of header
9  #ifndef SALESPERSON_H
10 #define SALESPERSON_H
11
12 class SalesPerson
13 {
14 public:
15
16     /// months in one year
17     static const int monthsPerYear = 12;
18     /// Constructor
19     SalesPerson();
20     /// Input sales from keyboard
21     void getSalesFromUser(void);
22     /// Set sales for a specific month
23     void setSales(int,double );
24     /// Summarize and print sales
25     void printAnnualSales(void);
26
27 private:
28
29     /// Prototype for utility function
30     double totalAnnualSales(void);
31     /// 12 monthly sales figures
32     double sales[monthsPerYear];
33 };
34
35 #endif
```

Class SalesPerson declares an array of 12 monthly sales figures (line 17) and the prototypes for the class's constructor and member functions that manipulate the array.

SalesPerson Class Implementation

```cpp
/* ******************************************************
 * Filename     :   SalesPerson.cpp
 * Author       :   Burak Kaleci
 * Date         :   10.10.2018
 * Description  :   SalesPerson Class Implementation
 * ******************************************************/

#include <iostream>     // for cout and cin
#include <iomanip>      // for setprecision and fixed

#include "SalesPerson.h"    // include definition of class SalesPerso

using namespace std;

/// Constructor initializes elements of array sales to 0.
SalesPerson::SalesPerson()
{
    for(int i=0;i<monthsPerYear;++i)
        sales[i]=0.0;
}

/// Get 12 sales figures from the user at the keyboard
void SalesPerson::getSalesFromUser(void)
{
    double salesFigure;

    for(int i=1;i<=monthsPerYear;++i){
        cout << "Enter sales amount for month " << i << ": ";
        cin >> salesFigure;
        setSales(i,salesFigure);
    }
}
```

```cpp
/// Set one of the 12 monthly sales figures
void SalesPerson::setSales(int month, double amount)
{
    if (month>=1 && month<=monthsPerYear && amount>0)
        sales[month-1]=amount;
    else
        cout << "Invalid month or sales figure" << endl;
}

/// Print total annual sales (with the help of utility function)
void SalesPerson::printAnnualSales(void)
{
    cout << setprecision( 2 ) << fixed << "\nThe total annual sales a
        << totalAnnualSales () << endl; // call utility function
}

/// Private utility function to total annual sales
double SalesPerson::totalAnnualSales(void)
{
    double total = 0.0; // initialize total
    for(int i=0;i<monthsPerYear;++i) // summarize sales results
        total += sales[ i]; // add month i sales to total

    return total;
}
```

SalesPerson Class Implementation

- The SalesPerson constructor (lines 16-20) initializes array sales to zero.
- The public member function setSales (lines 35-41) sets the sales figure for one month in array sales.
- The public member function printAnnualSales (lines 44-48) prints the total sales for the last 12 months.
- The **private utility function** totalAnnualSales (lines 51-58) totals the 12 monthly sales figures for the benefit of printAnnualSales.
- Member function printAnnualSales edits the sales figures into monetary format.

SalesPerson Class Driver Program

```
1  /* ********************************************************
2   * Filename     :   main.cpp
3   * Author       :   Burak Kaleci
4   * Date         :   10.10.2018
5   * Description  :   Driver program for class SalesPerson
6   * ********************************************************/
7
8  // include definition of class SalesPerson
9  #include "SalesPerson.h"
10
11 int main() {
12
13     SalesPerson s;
14
15     s.getSalesFromUser();
16     s.printAnnualSales();
17
18     return 0;
19 }
```

Notice that the driver's main function includes only a simple sequence of member-function calls. There are no control statements. The logic of manipulating the sales array is completely encapsulated in class Salesperson's member functions.

Constructors with Default Arguments

```
1  /* ********************************************************
2   * Filename     :   Time.h
3   * Author       :   Burak Kaleci
4   * Date         :   10.10.2018
5   * Description  :   An example program
6   * ********************************************************/
7
8  // prevent multiple inclusions of header
9  #ifndef TIME_H
10 #define TIME_H
11
12 // Time class definition
13 class Time
14 {
15 public:
16
17     /// Default Constructor
18     Time(int hour=0, int minute=0, int second=0);,
19     /// Set hour, minute and second
20     void setTime(int,int,int);
21     /// Set hour (after validation)
22     void setHour(int);
23     /// Set minute (after validation)
24     void setMinute(int);
25     /// Set second (after validation)
26     void setSecond(int);
27     /// Return hour
28     int getHour();
29     /// Return minute
30     int getMinute();
31     /// Return second
32     int getSecond();
33     /// Print time in universal-time format
34     void printUniversal(void);
35     /// Print time in standard-time format
36     void printStandard(void);
37
38 private:
39
40     int hour; // 0 - 23
41     int minute; // 0 - 59
42     int second; // 0 - 59
43 };
44
45 #endif
```

- Like other functions,constructors can specify default arguments.
- Line 18 declares the Time constructor to include default arguments,specifying a default value of zero for each argument passed to the constructor.
- Class Time provides set and get functions for each data member.
- The default arguments to the constructor ensure that, even if no values are provided in a constructor call, the constructor still initializes the data members.

Constructors with Default Arguments

```cpp
#include <iostream>     // for cout
#include <iomanip>      // for setfill and setw
#include <stdexcept>    // for invalid_argument exception class

#include "Time.h"   // include definition of class Time

using namespace std;

/// Time constructor initializes each data member to zero
Time::Time(int hour, int minute, int second )
{
    setTime( hour, minute, second ); // validate and set time
}
/// Set new Time value using universal time
void Time::setTime(int h, int m, int s)
{
    setHour(h);     // set private field hour
    setMinute(m);   // set private field minute
    setSecond(s);   // set private field second
}
/// Set hour value
void Time::setHour(int h)
{
    if (h >= 0 && h< 24 )
        hour =h;
    else
        throw invalid_argument( "hour must be 0-23" );
}
/// Set minute value
void Time::setMinute(int m)
{
    if (m >= 0 && m< 60 )
        minute =m;
    else
        throw invalid_argument( "minute must be 0-59" );
}
/// Set second value
void Time::setSecond(int s)
{
    if (s >= 0 && s< 60 )
        second =s;
    else
        throw invalid_argument( "second must be 0-59" );
}
```

```cpp
/// Return hour value
int Time::getHour()
{
    return hour;
}

/// Return minute value
int Time::getMinute()
{
    return minute;
}

/// Return second value
int Time::getSecond()
{
    return second;
}

/// print Time in universal-time format (HH:MM:SS)
void Time::printUniversal(void)
{
    cout << setfill('0') << setw( 2 ) << getHour() << ":"
        << setw( 2 ) << getMinute() << ":" << setw( 2 ) << getSecond();
}

// print Time in standard-time format (HH:MM:SS AM or PM)
void Time::printStandard(void)
{
    if (getHour() == 0 || getHour() == 12 )
        cout << 12;
    else
        cout << getHour() % 12;

    cout << ":" << setfill('0') << setw( 2 ) << getMinute() << ":" << setw( 2 )
        << getSecond();

    if (getSecond() < 12)
        cout << " AM";
    else
        cout << " PM";
}
```

Constructors with Default Arguments

- The Time constructor that receives values for parameters hour, minute and second that will be used to initialize private data members hour, minute and second,respectively.
- The Time constructor now calls setTime,which calls the setHour, setMinute and setSecond functions to validate and assign values to the data members.
- The default arguments to the constructor ensure that, even if no values are provided in a constructor call, the constructor still initializes the data members.
- Function setTime calls setHour to ensure that the value supplied for hour is in the range 0-23,then calls setMinute and setSecond to ensure that the values for minute and second are each in the range 0-59.
- Functions setHour, setMinute and setSecond each throw an exception if an out-of-range argument is received.

Constructors with Default Arguments

```
8  #include <iostream>
9  #include <stdexcept>
10 #include "Time.h"   // include definition of class Time
11
12 using namespace std;
13
14 int main()
15 {
16     Time t1; // all arguments defaulted
17     Time t2( 2 ); // hour specified; minute and second defaulted
18     Time t3( 21, 34 ); // hour and minute specified; second defaulted
19     Time t4( 12, 25, 42 ); // hour,minute and second specified
20
21     cout << "Constructed with:\n\nt1:all arguments defaulted\n";
22     t1.printUniversal(); // 00:00:00
23     cout << "\n";
24     t1.printStandard(); // 12:00:00 AM
25
26     cout << "\n\nt2: hour specified; minute and second defaulted\n";
27     t2.printUniversal(); // 02:00:00
28     cout << "\n";
29     t2.printStandard(); // 2:00:00 AM
30
31     cout << "\n\nt3: hour and minute specified; second defaulted\n";
32     t3.printUniversal(); // 21:34:00
33     cout << "\n";
34     t3.printStandard(); // 9:34:00 PM
35
36     cout << "\n\nt4: hour, minute and second specified\n";
37     t4.printUniversal(); // 12:25:42
38     cout << "\n";
39     t4.printStandard(); // 12:25:42 PM
40
41     // attempt to initialize t5 with invalid values
42     try
43     {
44         Time t5(27,74,99);
45     }
46     catch (invalid_argument &e)
47     {
48         cout << "\n\nException while initializing t5: " << e.what() << endl;
49     }
50
51     return 0;
52 }
```

- Function main initializes five Time objects
- One with all three arguments defaulted in the implicit constructor call.
- One with one argument specified.
- One with two arguments specified.
- One with three arguments specified.
- One with three invalid arguments specified.
- The program displays each object in universal-time and standard-time formats.
- For Time object t5, the program displays an error message because the constructor arguments are out of range.

Constructors with Default Arguments

- Time's set and get functions are called throughout the class's body.In particular,function setTime calls functions setHour, setMinute and setSecond,and functions printUniversal and printStandard call functions getHour, getMinute and getSecond.
- In each case, these functions could have accessed the class's private data directly.
- However,consider changing the representation of the time from three int values (requiring 12 bytes of memory)to a single int value representing the total number of seconds that have elapsed since midnight (requiring only four bytes of memory).
- If we made such a change, only the bodies of the functions that access the private data directly would need to change. In particular, the individual set and get functions for the hour, minute and second.
- There would be no need to modify the bodies of functions setTime, printUniversal or printStandard, because they do not access the data directly.
- Designing the class in this manner reduces the likelihood of programming errors when altering the class's implementation.

Constructors with Default Arguments

- Similarly,the Time constructor could be written to include a copy of the appropriate statements from function setTime.
- Doing so may be slightly more efficient,because the extra constructor call and call to setTime are eliminated.
- However,duplicating statements in multiple functions or constructors makes changing the class's internal data representation more difficult.
- Having the Time constructor call setTime and having setTime call setHour, setMinute and setSecond enables us to limit the changes to code that validates the hour, minute or second to the corresponding set function.
- This reduces the likelihood of errors when altering the class's implementation.

Destructors

- A destructor is another type of special member function.
- The name of the destructor for a class is the tilde character( ) followed by the class name.
- This naming convention has intuitive appeal, because the tilde operator is the bitwise complement operator, and, in a sense,the destructor is the complement of the constructor.
- A class's destructor is called implicitly when an object is destroyed. This occurs,for example, as an automatic object is destroyed when program execution leaves the scope in which that object was instantiated.

Destructors

- The destructor itself does not actually release the object's memory.
- It performs termination housekeeping before the object's memory is reclaimed, so the memory may be reused to hold new objects.
- Even though destructors have not been provided for the classes presented so far, every class has a destructor.
- If you do not explicitly provide a destructor, the compiler creates an "empty" destructor.